

Optimal Resizable Arrays 论文评述

张艺博 徐黄浩

2026 年 7 月 21 日

摘要

本文评述 Robert E. Tarjan 与 Uri Zwick 的论文 *Optimal resizable arrays*。该论文研究如何在保持数组随机访问效率的同时，降低动态扩缩容带来的额外空间开销。本文将围绕问题模型、相关工作、核心数据结构、复杂度证明以及方法局限展开分析。

目录

1	引言与论文定位	4
1.1	成员 A: 问题直觉与论文定位	4
1.2	成员 B: 补充与交叉审阅	5
2	问题定义与理论模型	5
2.1	成员 A: 补充与交叉审阅	5
2.2	成员 B: 问题定义与理论模型	5
2.2.1	可变长数组的抽象接口	5
2.2.2	内存块模型	6
2.2.3	两类空间开销	6
2.2.4	本文的复杂度目标	7
3	文献调研与相关工作	8
3.1	成员 A: 传统方法与工程背景	8
3.2	传统工作	8
3.2.1	其他传统工作	8
3.3	成员 B: 理论脉络与下界背景	9
3.3.1	Brodnik 等人的空间下界	9
3.3.2	Tarjan-Zwick 对下界视角的扩展	10
4	本文核心方法	10
4.1	成员 A: 整体构造与技术路线	10
4.2	整体构造与技术路线: 从空间下界到构造目标	10
4.3	成员 B: 空间优化机制	13
4.3.1	$r = 3$ 的两级块结构	13
4.3.2	一般 r 的分层块结构	14
4.3.3	访问时间与均摊更新时间	14
4.3.4	去除稳定空间中的 r 因子	15
5	正确性证明与理论分析	16
5.1	成员 A: 上界分析	16
5.1.1	上界分析: 一般 r 构造的均摊代价	16
5.1.2	访问与修改操作的最坏 $O(1)$ 时间	19
5.2	成员 B: 下界分析	22
5.2.1	标准实现及下界的适用范围	22

5.2.2	(N, k, ℓ) -growth game	22
5.2.3	二项式阈值与游戏下界	23
5.2.4	从真实结构到 $\Omega(r)$ 下界	24
6	技术直觉与方法评价	25
6.1	成员 A: 技术直觉与示例解释	25
6.1.1	技术直觉	25
6.1.2	示例解释	26
6.2	成员 B: 创新点、局限性与后续影响	26
6.2.1	核心创新与理论贡献	26
6.2.2	模型假设与定理边界	27
6.2.3	均摊更新与工程可行性	27
6.2.4	开放问题与后续意义	28
7	总结	28
7.1	成员 B: 总结	28

1 引言与论文定位

1.1 成员 A：问题直觉与论文定位

数组和可变数组是广泛应用的数据结构，可变数组的内存效率一直是研究者的研究热点。本文所研究的可变数组指的是，可以在末尾添加或者删除元素的数组[1]。为此，很自然想到，如何解决可变长数组的空间分配问题？一个典型场景是：内存已分配的空间已经被填满，此时需要插入新元素，却不能直接在原有内存块末尾追加，因为紧邻的内存可能已被其他对象占用。因此，必须重新申请一块更大的内存，将原有元素全部复制过去，再释放旧块。

也就是：如何维护一个可以增长和收缩的数组，使它既支持按下标 $O(1)$ 访问，又尽可能节省空间，同时 grow/shrink 操作仍然高效？

此问题在工程中与实际中用处广泛，比如：Java ArrayList，C++ vector。但是，作者认为，之前的设计（满了扩容、空了减小）过于基础，尽管它是最常用的数据结构之一，但是它的空间最优性没有完全解决。

这一问题可以从**机制**与**策略**两个层面加以分离与理解。

机制层面

1. 系统提供固定大小的存储块（block）；
2. 存储块可以存放实际数据，也可以存放指向其他块的指针；
3. Grow 操作：申请新块，必要时进行数据迁移；
4. Shrink 操作：删除末尾元素，释放不再需要的空块。

策略层面

1. 块的大小如何选取？
2. 使用多少层指针？Grow 过程中块的大小如何变化？
3. Shrink 的触发条件是什么？何时进行全局重建（rebuild）？

将机制与策略解耦，有助于清晰地分析和设计可变长数组的数据结构。

经典方法：翻倍策略（Doubling） 工业界广泛采用的成熟做法是几何增长策略，其核心规则为：当数组满时，分配一块大小为当前数组两倍的连续内存，并将所有元素复制过去。这一策略的摊还代价为 $O(1)$ ，随机访问同样为 $O(1)$ 最坏情况时间[2]。标准倍增算法可以达到 $O(N)$ （更确切说，应该是 $N + O(N)$ ）空间开销和 $O(1)$ 的时间开

销，但其最坏时刻的空间浪费可达 50%。进一步，我们可以将这些策略总结成如下规律(再后面第三节传统文献调研部分有详细说明)：假设每次数组满后，增长因子为 $(1 + \alpha)$ ，那么新数组刚扩展完毕时浪费率为 $\frac{\alpha}{1+\alpha}$ ，但是， α 也不能无限小，当其越小时，时间开销越大，因为平均复制次数增多了，假设数组容量为 C 后触发 grow，新容量变为 $C' = (1 + \alpha)C$ ，每个操作的成本为 $1/\alpha$

本文的目标 能否将空间占用压缩至 $N + o(N)$ ，同时仍然保持 $O(1)$ 的随机访问时间？这正是 Tarjan 和 Zwick 论文试图回答的核心问题[1]。

核心思想 作者认为应该区分两种空间，第一种是存储空间：数组当前大小为 N ，平时为了存储它并支持访问，需要多少空间？第二种是resize 临时空间：grow/shrink 的过程中，短暂需要多少额外空间？

1.2 成员 B：补充与交叉审阅

待补充。

2 问题定义与理论模型

2.1 成员 A：补充与交叉审阅

待补充。

2.2 成员 B：问题定义与理论模型

本节给出本文所讨论的可变长数组（resizable array）的形式化定义，并说明 Tarjan 和 Zwick 论文中用于衡量空间开销的模型[1]。与普通动态序列不同，本文研究的对象更接近栈式数组：元素只能在数组末端增加或删除，但任意位置的读取与修改仍然需要保持常数时间。

2.2.1 可变长数组的抽象接口

设当前数组为 A ，长度为 N 。论文中的可变长数组支持以下基本操作：

1. `Array()`：创建并返回一个初始为空的数组；
2. `Length()`：返回数组当前长度；
3. `Get(i)`：返回下标为 i 的元素，其中 $0 \leq i < N$ ；

4. **Set(i, a)**: 将下标为 i 的元素修改为 a , 其中 $0 \leq i < N$;
5. **Grow(a)**: 在数组末尾加入一个新元素 a , 使数组长度从 N 变为 $N + 1$;
6. **Shrink()**: 删除数组末尾元素, 使数组长度从 N 变为 $N - 1$ 。

这里最重要的限制是：插入和删除只发生在末端。若允许在任意位置插入或删除，则后续元素的下标都可能发生变化，问题会转化为动态序列维护。该类问题即使不要求极端空间效率，也通常需要更复杂的数据结构和更高的操作复杂度。因此，本文刻意将研究对象限定在末端扩缩容的数组模型上。

2.2.2 内存块模型

论文假设底层内存管理系统允许申请和释放任意固定长度的数组块。一个可变长数组的具体实现由若干动态分配的块组成，主要包括三类：

1. **数据块 (data block)**: 存放数组中的实际元素；
2. **索引块 (index block)**: 存放指向数据块或其他索引块的指针；
3. **辅助块 (auxiliary block)**: 存放块数量、块长度、层级计数器等辅助信息。

这种模型抽象了实际系统中的动态内存分配行为。若数组必须始终存放在单个连续内存块中，那么扩容时通常只能重新申请更大的块并复制所有元素；而本文允许数组被分散存放在多个块中，再通过索引块维持常数时间访问。这也是后续分层结构能够降低空间浪费的基础。

2.2.3 两类空间开销

本文的关键建模创新是区分两类空间：

1. **存储与访问空间**: 当数组处于稳定状态、长度为 N 时，为了存储所有元素并支持 **Get** 和 **Set** 所需的空間；
2. **扩缩容临时空间**: 执行 **Grow** 或 **Shrink** 时，在元素移动、块合并或块拆分过程中短暂额外占用的空间。

用论文中的记号表示，若一个实现称为 $(s(N), t(N))$ -implementation，则表示它满足[1]：

$$\text{稳定存储空间} \leq N + s(N),$$

并且在扩缩容操作过程中最多使用：

$$\text{临时峰值空间} \leq N + t(N).$$

这里的 $s(N)$ 衡量平时多出来的空间，例如索引指针、空闲块和辅助信息； $t(N)$ 衡量一次 grow 或 shrink 过程中可能出现的峰值空间。传统讨论往往把这两种空间混在一起计算，而 Tarjan 和 Zwick 的核心观察是：在很多应用中，平时占用空间比扩缩容瞬间的临时空间更值得优化。例如，一个程序可能同时维护许多可变长数组，但任意时刻只有少数数组正在扩容或缩容。

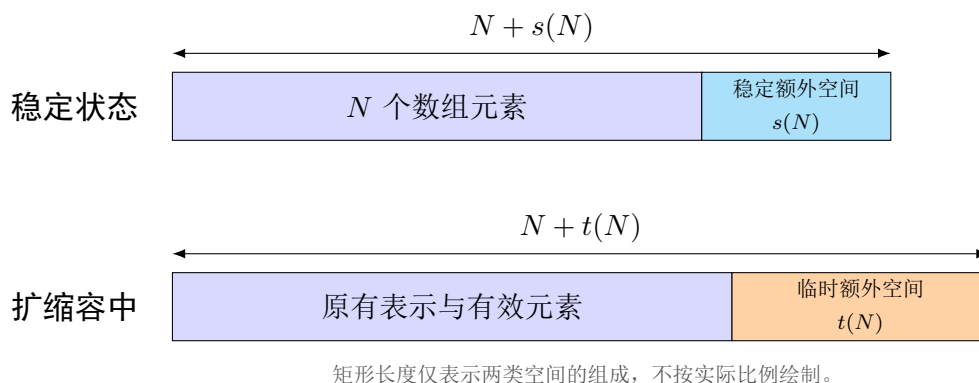


图 1: 稳定空间与扩缩容临时空间的区分（根据文献 [1] 整理）。

2.2.4 本文的复杂度目标

在上述模型下，本文希望同时达到以下目标：

1. Get 和 Set 操作保持 $O(1)$ 最坏情况时间；
2. Grow 和 Shrink 操作保持较低的均摊时间；
3. 稳定存储空间尽量接近信息论意义上的下限 N ；
4. 在允许一定临时空间的前提下，突破传统 $N + O(\sqrt{N})$ 空间界限。

论文最终给出的结果是一条由参数 r 控制的折中曲线：对于任意整数 $r \geq 2$ ，可以将稳定存储空间压到 $N + O(N^{1/r})$ ，同时扩缩容时允许短暂使用 $N + O(N^{1-1/r})$ 空间，并保持访问 $O(1)$ 最坏情况时间、扩缩容 $O(r)$ 均摊时间。后文对核心结构和下界证明的分析，都是围绕这组时间–空间折中展开的。

3 文献调研与相关工作

3.1 成员 A：传统方法与工程背景

在进行相关工作阐述之前，我们先解释一下摊还分析(Amortized Analysis)。

摊还分析是一种分析操作序列平均代价的技术，由于难以衡量单次操作的最坏代价，所以我们从总代价的角度出发，使用总代价除以操作次数，得到平均每次操作花费代价。

具体而言，有三种分析方法：

1. 聚合分析：直接计算总代价，除以操作次数 2. 记账法 3. 势能法

本文中使用的主要是第一种方法。

此外，本文也使用了记账法，记账法的原理是分配 credit 并记录每次消耗，后文会详细阐述。

3.2 传统工作

首先介绍论文中提到的相关工作

朴素方法 很自然想到，最直接实现可变长数组的方案就是维护一个大小恰好等于当前元素数 N 的固定数组，每次 grow 或者 shrink 时都触发全量重建

它的存储空间显然是 $N + O(1)$ ，可以达到理论最优，但是 grow 或者 shrink 的摊还代价为 $\Theta(N)$ ，并且临时空间最大为 $2N + O(1)$

Geometric Expansion 第二种传统方法就是前文提到过的几何增长策略，用空间换取时间，通过预分配额外容量降低重建成本。它的具体方法如下：

假设增长因子为 $1 + \alpha$ ，初始分配大小为 C 的数组，当 N 个元素填满后，分配大小为 $(1 + \alpha)N$ 的新数组，复制全部元素，当元素数降至 $\frac{N}{1+\alpha}$ 以下时，收缩至 $\frac{N}{1+\alpha}$

使用摊还分析该数据结构，它的存储空间为 $O(N)$ ，grow 摊还代价为 $N/(\alpha N) = 1/\alpha$

可以看到，如果我们增大 α ，虽然摊还代价小了，但是我们总的存储空间增加，因此，传统方法的局限就在于此，难以平衡空间和时间。根本原因在于粒度太粗，也就是现在的空间划分只是计算的平均空间，后续作者针对这种不足提出了本文的改进。

3.2.1 其他传统工作

这里补充论文未详细讨论、但与动态数组设计有关的几类工作。工程中的连续动态数组通常采用几何扩容，不过具体增长因子由标准库实现和版本决定，并不是 C++、Java 或 Python 语言规范统一规定的常数。无论因子如何选择，这类方法的共同点都是保留线性级别的空闲容量，以换取 $O(1)$ 均摊追加时间[2]。

若改用固定大小为 B 的分块链表，并假设除尾块外的数据块均填满，则块指针和尾部空位带来的额外空间为 $O(N/B + B)$ 。但是，在没有额外目录的情况下，按下标访问需要顺序经过 $O(N/B)$ 个块，不能满足本文要求的 $O(1)$ 最坏情况访问。Bagwell 的 VList 将元素放入按几何级数增长的块中，以避免每次增长都整体复制旧元素，并利用块大小的规律支持高效索引[3]。这类结构说明，分块和几何层次可以同时改善增长与访问，但也会引入多块管理和间接寻址开销。

另一条研究路线考虑比本文更一般的 rank-based sequence，即允许在任意秩位置插入或删除元素。Goodrich 和 Kloss 提出的 tiered vector 以多层循环数组组织序列[4]；Bille 等人随后进一步给出了访问、更新和额外空间之间可调的动态数组结构，并进行了实现评估[5]。这些结果与本文都使用分层分块思想，但操作模型不同：前者重点处理任意位置更新，本文则只允许在末端执行 Grow 和 Shrink，并进一步区分稳定空间与扩缩容临时空间。

除渐近界之外，Joannou 和 Raman 对多种 extendible array 进行了实证比较，考察其运行时间、空间占用和缓存行为[6]。这项工作提示我们，较小的渐近额外空间并不必然转化为更快的实际程序；分配次数、局部性和索引层数同样会影响工程表现。

3.3 成员 B：理论脉络与下界背景

本节梳理可变长数组空间效率问题的前置工作。传统动态数组通过几何扩容获得 $O(1)$ 均摊更新时间，但会保留线性级别的空闲容量[2]；Sitarski 的 hashed array tree 和 Brodnik 等人的结构则把稳定状态下的额外空间降低到 $O(\sqrt{N})$ [7, 8]。Tarjan 和 Zwick 的工作建立在这条脉络上，但其关键贡献不是简单改进已有结构，而是重新区分“稳定存储与访问空间”和“扩缩容临时空间”[1]。

3.3.1 Brodnik 等人的空间下界

Brodnik 等人的经典结果说明，在不区分临时空间的模型中， $N + \Omega(\sqrt{N})$ 级别的空间占用是不可避免的[8]。这个下界的证明思路很简洁，也解释了为什么后续工作很难在原有模型下突破 $\sqrt{N}$ 壁垒。

考虑只执行 N 次 Grow 操作后的状态。设此时数组元素分布在 k 个连续内存块中。由于这些块共同保存 N 个元素，它们的总大小至少为 N 。同时，为了之后能够访问每个块，数据结构至少需要维护到这些块的指针或等价索引信息。

如果 $k \geq \sqrt{N}$ ，那么仅指针数量就已经造成 $\Omega(\sqrt{N})$ 的额外空间。因此总空间至少为

$$N + \Omega(\sqrt{N}).$$

如果 $k < \sqrt{N}$ ，那么在保存 N 个元素的这些块中，必然存在一个大小超过 $\sqrt{N}$ 的块。设这个大块是在数组长度为 $N' \leq N$ 时被分配的。它刚被分配出来时还没有装入

原有元素，而旧的 N' 个元素仍然存放在其他地方，所以当时的总空间会超过

$$N' + \sqrt{N'}.$$

因此，无论块数多还是少，某个时刻都会出现 $N + \Omega(\sqrt{N})$ 级别的空间占用。直观地说，块多会带来指针开销，块少则会带来大块分配时的临时空间峰值。

3.3.2 Tarjan–Zwick 对下界视角的扩展

Tarjan 和 Zwick 并不是否定 Brodник 等人的下界，而是指出该下界把两类空间混在了一起：一类是平时存储和访问数组需要的空间，另一类是执行 **Grow** 或 **Shrink** 时短暂出现的临时空间[1]。若用 $(s(N), t(N))$ -implementation 描述二者，则平时总空间为 $N + s(N)$ ，扩缩容过程中的峰值空间为 $N + t(N)$ 。

在这个模型下，Brodник 下界可以推广为乘积形式。仍然考虑执行 N 次 **Grow** 后的状态。由于每一个数据块都必须可达，块数不能超过额外索引空间的量级，因此可以认为块数至多为 $O(s(N))$ 。这些块一共存放 N 个元素，所以至少有一个块的大小为

$$\Omega\left(\frac{N}{s(N)}\right).$$

当这个大块刚被分配出来时，它尚未存放旧元素，旧元素仍然需要由原有块保存。因此在那次 **Grow** 的过程中，临时额外空间至少达到该大块的大小量级，即

$$t(N) = \Omega\left(\frac{N}{s(N)}\right).$$

于是得到

$$s(N)t(N) = \Omega(N).$$

忽略常数后，这就是论文中 $s(N)t(N) \geq N$ 的含义。它表明平时额外空间和扩缩容临时空间不能同时任意小。若希望稳定存储空间为 $N + O(N^{1/r})$ ，则临时空间至少需要达到 $N + \Omega(N^{1-1/r})$ 。Tarjan 和 Zwick 后续给出的构造正好达到这一量级，因此它在两类空间的 trade-off 上是最优的。

4 本文核心方法

4.1 成员 A：整体构造与技术路线

4.2 整体构造与技术路线：从空间下界到构造目标

在进入本文的具体数据结构构造之前，作者首先从已有的下界出发，重新审视 resizable array 问题中的空间开销。直观上，Brodник 等人的数据结构已经给出了 $N +$

$O(\sqrt{N})$ 大小的空间使用，这暗示在平时存储空间和扩容过程中的空间开销之间， $\sqrt{N}$ 可能是某种最优平衡。本文的关键观察是：这个下界中混合了两类不同的空间。一种是数据结构在稳定状态下为了存储并访问数组所需要的空间；另一种是 grow 或 shrink 过程中短暂出现的临时空间。作者正是通过区分这两类空间，重新打开了 $N + O(\sqrt{N})$ 这一传统界限。

Theorem 4.1. Brodnik 等人的下界可以表述为：任何维护 resizable array 的数据结构，在某些时刻都必须使用 $N + \Omega(\sqrt{N})$ 空间。这个结论即使只考虑 grow 和 access 操作也成立[8, 1]。

其证明思路可以理解为对 block 数量进行分类讨论。假设经过 N 次 grow 操作后，数组中有 N 个元素，这些元素被存放在若干个动态分配的连续内存块中。设当前一共有 k 个内存块。

第一种情况是 $k \geq \sqrt{N}$ 。由于每一个内存块都必须能够被数据结构访问到，因此每个块至少需要一个指针指向它。这样，仅指针所需要的空间就至少为 k 。而原始数据本身至少需要 N 个空间，所以总空间至少为

$$N + k \geq N + \sqrt{N}.$$

这说明当 block 数量较多时，额外空间主要来自指针开销。也就是说，这一部分对应的是平时为了组织和访问这些 block 而长期存在的存储空间。

第二种情况是 $k < \sqrt{N}$ 。此时， N 个元素被放入少于 $\sqrt{N}$ 个 block 中。根据抽屉原理，必然存在一个 block 的大小大于 $\sqrt{N}$ 。设这个大 block 是在某一次 grow 操作中被申请出来的，当时数组长度为 $N' \leq N$ 。在该 block 刚刚申请出来的瞬间，它还没有真正承载原来的元素，而原来的 N' 个元素仍然存放在旧的 block 中。因此，在这一瞬间，除了旧的 N' 个元素空间之外，还额外出现了一个大小超过 $\sqrt{N'}$ 的新 block，于是当时总空间至少为

$$N' + \Omega(\sqrt{N'}).$$

这说明当 block 数量较少时，必然存在某个很大的 block，而这个大 block 在申请时会产生显著的临时空间。

因此，Theorem 4.1 的两个 case 实际上分别对应两类空间来源：case 1 说明的是平时存储和访问所需的指针空间；case 2 说明的是某一时刻突然申请一个大 block 时产生的临时空间。这也是本文区分平时空间和额外临时空间的原因。

Theorem 4.2. 在 Theorem 4.1 的基础上，作者进一步给出了更一般的乘积下界。对于任意 $(s(N), t(N))$ -implementation，作者证明了如下乘积下界[1]：

$$s(N)t(N) \geq N.$$

其中, $s(N)$ 表示平时存储数组时允许的额外空间, $t(N)$ 表示 grow 或 shrink 过程中允许的临时额外空间。

这个证明与 Theorem 4.1 的思路保持一致, 仍然可以从抽屉原理出发。若数据结构在平时只允许 $N + s(N)$ 的空间, 那么除了 N 个真实元素之外, 最多只能有 $s(N)$ 量级的额外空间用于指针、索引或 block 管理信息。特别地, 由于每个 block 至少需要被某个指针或索引结构访问到, 我们可以认为当前 block 的数量至多为 $s(N)$ 量级。

于是, N 个元素分布在至多 $s(N)$ 个 block 中, 必然存在一个 block 的大小不小于

$$\frac{N}{s(N)}.$$

当这个 block 被申请出来时, 它尚未装入旧元素, 而旧元素仍然存放在原来的 block 中。因此, 在这一瞬间, resize 过程至少需要额外容纳这样一个大小为 $N/s(N)$ 量级的 block。于是有

$$t(N) \geq \frac{N}{s(N)}.$$

进一步得到

$$s(N)t(N) \geq N.$$

这个结论可以看成 Theorem 4.1 的加强版本。因为在 Theorem 4.1 中, 若取 $s(N) = t(N) = \sqrt{N}$, 就正好得到原来的 $N + \Omega(\sqrt{N})$ 下界。更一般地, 如果我们希望平时的额外存储空间达到

$$s(N) = O(N^{1/r}),$$

那么由乘积下界可知, resize 过程中就不可避免地需要

$$t(N) \geq \Omega(N^{1-1/r}).$$

这说明, 如果我们想把稳定状态下的空间压缩到 $N + O(N^{1/r})$, 就必须接受 grow 或 shrink 过程中短暂使用 $N + O(N^{1-1/r})$ 的临时空间。

由下界引出的构造目标。 以上两个定理给出了本文整体构造的理论目标: 作者希望围绕乘积下界 $s(N)t(N) \geq N$ 给出相匹配的数据结构。换言之, 既然下界已经说明

$$s(N) = O(N^{1/r}) \implies t(N) = \Omega(N^{1-1/r}),$$

那么接下来的目标就是构造一个数据结构, 使其真正达到

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}).$$

因此, 本文后续构造部分的基本路线可以概括为: 先通过下界区分平时空间和临时空间, 然后围绕这个下界给出相匹配的结构。构造方面, 最基本的 $r = 2$ 情况已经由

Brodnik 等人的 $N + O(\sqrt{N})$ 结构给出；随后作者给出特殊的 $r = 3$ 构造，再进一步推广到一般的 r 。

总结一下：这部分先说明 $s(N)t(N) \geq N$ 的理论最优，再通过 $r = 3$ 的例子展示如何突破稳定状态下的 $N + O(\sqrt{N})$ 空间，最后给出一般 r 的多层 block 构造。

4.3 成员 B：空间优化机制

本节进一步解释上述分层构造中的空间优化机制。核心思想不是消除扩缩容时的数据搬移，而是把空间开销区分为稳定状态下的额外空间和扩缩容过程中的临时空间，并有意识地把较大的空间需求限制在短暂的块合并、拆分和重建过程中[1]。

4.3.1 $r = 3$ 的两级块结构

论文首先讨论一个 $(O(N^{1/3}), O(N^{2/3}))$ -implementation[1]。令参数 B 满足

$$N^{1/3} \leq B \leq 4N^{1/3}.$$

数据结构使用大小为 B^2 的大块存储数组中已经稳定下来的部分，并使用至多 $2B$ 个大小为 B 的小块保存数组末端。所有大块始终保持填满；小块中至多有一个部分填充的块，并可以额外保留一个空块。两类数据块分别由两个索引块管理。

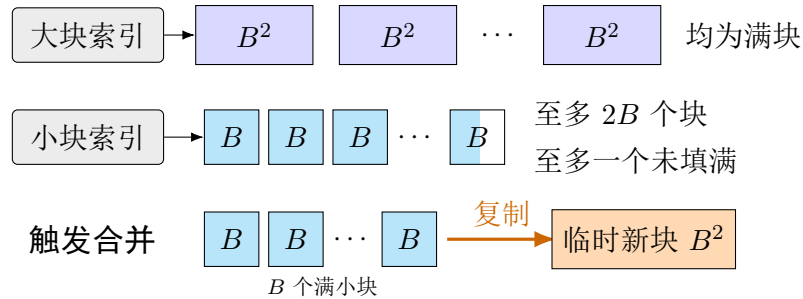


图 2: 参数 $r = 3$ 时的大块、小块与合并过程（根据文献 [1] 重绘）。

这一设计可以看作对 hashed array tree 的再次分层。如果直接使用大小约为 $N^{2/3}$ 的数据块，最后一个数据块可能只存有少量元素，稳定状态下会浪费 $O(N^{2/3})$ 空间。现在改用大小约为 $N^{1/3}$ 的小块表示末端尚未填满的部分，尾部浪费和索引开销都只需 $O(N^{1/3})$ 空间。

这种压缩并非没有代价。当 $2B$ 个小块均已填满时，结构会将其中 B 个小块的元素复制到一个新申请的、大小为 B^2 的大块中；当小块耗尽而数组继续执行 **Shrink** 时，则需要把一个大块拆成 B 个小块。因此，合并或拆分过程中会短暂使用

$$O(B^2) = O(N^{2/3})$$

的额外空间。由此得到的正是“较小稳定空间、较大临时空间”的折中：平时只保留 $O(N^{1/3})$ 额外空间，而 `resize` 过程允许 $O(N^{2/3})$ 的临时峰值。

4.3.2 一般 r 的分层块结构

对于任意整数 $r \geq 2$ ，第 6 节把两级结构推广为 $r-1$ 个层级[1]。结构维护参数 B ，使得

$$N^{1/r} \leq B < 4N^{1/r},$$

并使用大小为

$$B, B^2, \dots, B^{r-1}$$

的数据块。大小超过 B 的块始终填满；最低层大小为 B 的块中，至多最后一个没有填满。每种块大小对应一个索引块，因此一共维护 $r-1$ 个索引块。

各层块的数量可以理解为 N 的一种冗余 base- B 表示。若只支持 `Grow`，每层保留至多 B 个块即可：当某层积累了 B 个大小为 B^i 的满块时，将它们合并成一个大小为 B^{i+1} 的块。为了同时支持 `Shrink`，论文把每层的容量放宽到至多 $2B$ 个块。当最低层没有块可供删除时，结构找到最近的非空高层，并将一个高层块逐级拆成更小的块。这个冗余区间使合并与拆分之间留有缓冲，避免连续操作反复触发相反方向的重构。

稳定状态下，数据块除最低层的最后一个块外均为满块，主要额外开销来自各层索引、计数器和少量未使用位置。每层索引规模为 $O(B)$ ，共有 $r-1$ 层，故稳定额外空间为

$$O(rB) = O(rN^{1/r}).$$

一次合并所申请的最大新块大小为 B^{r-1} ，因此扩缩容过程中的额外空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

第 6 节由此得到

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation.}$$

4.3.3 访问时间与均摊更新时间

分层并不意味着按下标访问必须逐层查找。论文令 B 为 2 的幂，并把各层块数压缩为机器字中的冗余 base- B 计数器。借助 `msb`、`lsb`、位掩码和移位操作，可以在 word-RAM 模型下以 $O(1)$ 最坏情况时间确定目标元素所在的层、数据块及块内偏移。因此，`Get` 和 `Set` 仍保持常数时间[1]。

合并与拆分虽然可能移动整个块，但其代价可以分摊到一段操作序列上。每个新元素先进入最低层，随后至多沿 $r-1$ 个层级向上移动；拆分的代价也可以由两次同级拆分之间积累的 `Shrink` 操作支付。因此，`Grow` 和 `Shrink` 的均摊代价为 $O(r)$ 。本章只保留这一结构直觉，完整的 credit 分析和相应下界将在后续理论分析中讨论。

4.3.4 去除稳定空间中的 r 因子

当 r 是常数时, $O(rN^{1/r})$ 与 $O(N^{1/r})$ 在渐近意义下没有区别; 但当 $r = r(N)$ 随 N 增长时, 第 6 节结果中的 r 因子不能忽略。第 7 节为此提出 data structuring transformation。它针对一类 standard implementation: 临时空间峰值主要出现在申请新块, 并按顺序把若干旧块内容复制到新块的过程中。

设一个标准结构的参数原本为

$$(s(N), O(N)),$$

并给定非递减阈值函数 $t(N)$ 。当它试图申请长度超过 $t(N)$ 的新块时, 不再申请单个完整大块, 而是顺序申请若干个大小至多为 $t(N)$ 的小块, 并保存指向这些小块的额外指针。这样, 任一时刻新申请而尚未填满的空间被控制在 $O(t(N))$; 另一方面, 拆分大块所需的额外指针数为 $O(N/t(N))$ 。Lemma 7.2 给出的转换结果为[1]

$$\left(s(N) + O\left(\frac{N}{t(N)}\right), s(N) + O(t(N))\right)\text{-implementation,}$$

同时不增加访问操作的渐近最坏情况时间, 也不增加扩缩容操作的渐近均摊时间。

为了得到最终界, 论文取

$$t(N) = N^{1-1/r},$$

并把该转换应用于第 6 节中参数为 $2r$ 的标准结构。输入结构的稳定额外空间为

$$O(rN^{1/(2r)}).$$

Theorem 7.3[1] 所处理的参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}$$

在此范围内, 上述空间项以及新增的

$$O(N/t(N)) = O(N^{1/r})$$

指针空间都可被 $O(N^{1/r})$ 吸收。最终得到

$$(O(N^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

并继续支持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。这个转换说明, 论文的最终空间界并非只来自增加层数, 而是来自“分层表示”与“限制单次物理分配规模”两步优化的结合。

5 正确性证明与理论分析

5.1 成员 A：上界分析

5.1.1 上界分析：一般 r 构造的均摊代价

对于一般 r 的构造：

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation,}$$

证明 grow 和 shrink 的均摊成本均为 $O(r)$ 。

这里的一般构造指的是（实际上上文已经阐明，这里简单提及）：令 $B \approx N^{1/r}$ ，数据被存放在大小为

$$B, B^2, \dots, B^{r-1}$$

的多层 blocks 中。每一层最多维护 $2B$ 个 blocks。于是，从空间角度看，每一层的 index block 大小为 $O(B)$ ，一共有 $r - 1$ 层，因此稳定状态下的额外存储空间为

$$O(rB) = O(rN^{1/r}).$$

另一方面，resize 过程中最大的临时申请发生在分配一个大小为 B^{r-1} 的 block 时，因此临时空间为

$$O(B^{r-1}) = O(N^{1-1/r}).$$

所以该结构首先满足

$$(O(rN^{1/r}), O(N^{1-1/r}))\text{-implementation}$$

的空间要求。

下面主要说明 grow/shrink 的均摊代价。前面的构造其实已经隐含了这个证明思想：因为最多有 $r - 1$ 层，所以对于任意一个元素，它最坏也只会在这几层之间被移动 $O(r)$ 次。换句话说，从直观上看，总的元素移动代价至多为 $O(Nr)$ ，而因为共有 N 个元素，均摊到每个元素上就是 $O(r)$ 。这是一个直觉证明，更加严谨的证明使用了记账法。

Grow 的均摊代价。 论文将一次 Grow 或 Shrink 的成本定义为 item assignments 的次数。对于 Grow 来说，普通插入一个新元素的成本为 1；如果触发 Combine-Blocks，则还要计算合并过程中元素复制的成本。

作者为 level i 中的每个元素分配

$$r - i$$

个 credit。这里 level i 指的是元素当前位于大小为 B^i 的 block 中。新插入的元素总是先进入 level 1，因此初始获得

$$r - 1$$

个 credit。当某个元素因为 Combine-Blocks 从 level i 被复制到 level $i + 1$ 时，它的 credit 从

$$r - i$$

变为

$$r - (i + 1),$$

正好减少 1，而这个减少的 credit 就用来支付这一次元素复制的代价。

因此，每个元素从 level 1 一路移动到更高层的过程中，credit 始终非负，且每一次复制都由 credit 支付。新插入元素时需要额外分配 $r - 1$ 个 credit，再加上插入本身的常数成本，所以 Grow 的均摊成本为

$$O(r).$$

这也对应了论文中“Combine-Blocks 的摊还成本为 0”的结论。

Shrink 的均摊代价。 Shrink 的分析比 Grow 更细一些，因为 Shrink 并不是简单地反向使用上面的元素 credit。我们先回忆一下 shrink 的操作。Shrink 始终删除数组末尾元素，也就是最新一段中最后的元素。如果最后一个 size- B block 中还有元素，那么直接删除即可，不需要复制元素，因此 item assignment 成本为 0。但是如果当前没有 size- B block，也就是 $n_1 = 0$ ，就必须调用 Split-Blocks，从更高层拆出一些低层 blocks，然后再删除末尾元素。

因此 Shrink 分为两种情况：

case 1: 最后一个 size- B block 还有元素，直接删除，代价为 0；

case 2: 没有 size- B block，需要进行一次 Split-Blocks。

设这次 Split-Blocks 发生在 level k ，也就是当前最小的非空层为 k 。此时算法会将最后一个大小为 B^k 的 block 拆开。这个 size- B^k block 被拆成

拆分后，levels 1 到 $k - 1$ 中一共正好包含 B^k 个元素。size- B^k block 会被拆成更低层的 blocks：对于 level $k - 1, k - 2, \dots, 2$ ，每一层各生成 $B - 1$ 个对应大小的 blocks，也就是 $B - 1$ 个 size- B^{k-1} blocks， $B - 1$ 个 size- B^{k-2} blocks，依次直到 $B - 1$ 个 size- B^2 blocks；最后在 level 1 生成 B 个 size- B blocks。也就是说，一次 split 大致可以理解为把一个 B^k 级别的大块向下拆成若干层小块，从而重新暴露出可以继续 shrink 的 size- B blocks。关键观察是：两次同级别的 level k split 之间，至少要经历 B^k 次 Shrink。原因是，一次 level k split 之后，低层 $1, \dots, k - 1$ 中已经有 B^k 个元素；如果下一次还想再次发生 level k split，就必须先把这些低层元素全部删掉。因此，中间至少发生 B^k 次

Shrink。论文还指出，如果前一次不是同级 split，而是一次 Combine-Blocks 创建了一个新的 size- B^k block，那么在该 combine 之后，levels 1 到 $k-1$ 中也至少有 B^k 个元素，所以到下一次 level k split 之间同样至少有 B^k 次 Shrink。

为了支付 Split-Blocks 的成本，作者给每一次 Shrink 操作额外分配 level credit：每次 Shrink 都给每一层增加 3 个 credit。因此，如果一次 level k split 发生，那么在这之前 level k 至少已经积累了

$$3B^k$$

个 credit。

以上是shrink获得的，现在分析成本，成本包括两部分：

第一部分是实际复制成本。拆开一个 size- B^k block，需要复制其中的 B^k 个元素，因此实际 item assignment 成本为

$$B^k.$$

第二部分是给移动后的元素补充 credit。因为 Split-Blocks 会把原来在 level k 的元素移动到更低层，而低层元素未来可能还会被向高层复制，所以需要补充元素 credit。若一个元素从 level k 被移动到 level i ，其中 $i < k$ ，那么它需要额外补充的 credit 数量为

$$k - i.$$

根据 split 后各层元素的分布，需要补充的 credit 总量为

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i.$$

其中第一项和求和式共同对应 level 1 中的 B^2 个元素，其余项对应 level 2 到 level $k-1$ 中的元素。

接下来对该式进行上界估计：

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i)B^i \leq B \sum_{i=1}^{k-1} (k-i)B^i.$$

令 $j = k - i$ ，则有

$$B \sum_{i=1}^{k-1} (k-i)B^i = B \sum_{j=1}^{k-1} jB^{k-j} \leq B^{k+1} \sum_{j \geq 1} jB^{-j}.$$

利用无穷级数

$$\sum_{j \geq 1} jB^{-j} = \frac{B}{(B-1)^2},$$

可得

$$B^{k+1} \sum_{j \geq 1} jB^{-j} = \frac{B^{k+2}}{(B-1)^2} \leq 2B^k,$$

其中最后一步使用了 $B \geq 4$ 。

因此，一次 level k 的 Split-Blocks 所需支付的总成本不超过

$$B^k + 2B^k = 3B^k.$$

而在该 split 发生之前，level k 已经至少积累了 $3B^k$ 个 credit，所以这些 credit 足以支付本次 split 的实际复制成本以及移动元素后的 credit 补充。因此，Split-Blocks 的摊还成本为 0。

Rebuild 的成本。 最后还需要考虑 Rebuild。Grow 或 Shrink 都可能导致参数 B 不再适合当前的 N ，因此算法会在 $N = B^r$ 时将 B 翻倍，或在 $N = (B/4)^r$ 时将 B 减半并重建结构。论文中同样使用 credit 支付 Rebuild 成本：每次 Grow 或 Shrink 再额外给整个数据结构增加 $2r$ 个 credit。这样虽然会把每次操作的均摊成本再增加 $O(r)$ ，但仍然保持在 $O(r)$ 范围内。

当 Rebuild 发生时，两次 Rebuild 之间已经经过了线性数量的 Grow 或 Shrink 操作，因此数据结构已经积累了足够多的 credit。Rebuild 的实际成本为 N ，而重建后为所有元素重新分配 credit 的总量至多为 $(r-1)N$ ，因此积累的全局 credit 足以支付 Rebuild 的成本和重新分配 credit 的成本。所以 Rebuild 的摊还成本也可以视为 0。

所以，只有 Grow 和 Shrink 均摊成本为 $O(r)$ （相当于主动增加 r 个 credit）其他操作完全可以由这些支付。

5.1.2 访问与修改操作的最坏 $O(1)$ 时间

对于访问与修改操作的最坏 $O(1)$ 时间，首先有一个直接的朴素思路：从高层到低层扫描，判断待访问元素落在哪一层。由于一般 r 的构造中最多有 $r-1$ 层，这样做的复杂度为 $O(r)$ 。这一方法虽然直观，但还不能达到论文所要求的 worst-case $O(1)$ 。因此，作者进一步利用了广义 B 进制表示，将“寻找元素所在层”这一过程压缩到常数时间。

具体来说，令 n_0 表示最后一个部分填充的 size- B block 中的元素个数；令 n_1 表示 full size- B blocks 的数量；更一般地， n_j 表示 full size- B^j blocks 的数量。由于每一层的 block 数量都不超过 $2B$ ，因此有 $n_j \leq 2B$ 。于是当前数组长度可以写成一种冗余的 base- B 表示：

$$N = \sum_{j=0}^{r-1} n_j B^j.$$

这里之所以说是冗余表示，是因为普通 base- B 表示中每一位应小于 B ，而这里每个 n_j 可以达到 $2B$ 。不过这并不会破坏访问过程，反而正是 redundant base- B counter 的体现。

接下来定义

$$N_k = \sum_{j=0}^{k-1} n_j B^j.$$

直观上, N_k 表示低于 level k 的那些 blocks 中一共存放了多少元素。由于本文的数据结构中较高层 blocks 存放数组前面的、较稳定的元素, 而较低层 blocks 存放靠近末尾的新元素, 因此判断一个元素在哪一层, 可以等价地从数组末尾往前数。

设要访问的元素下标为 i , 其中 $0 \leq i < N$ 。令

$$x = (N - 1) - i.$$

这里 x 表示该元素从数组末尾倒数的位置。于是, 第 i 个元素位于 level k 当且仅当

$$N_k \leq x < N_{k+1}.$$

这个式子是访问证明中的核心。它说明, 一个元素在哪一层, 并不是由前面有多少元素决定, 而可以由它后面有多少元素决定; 而从后往前看时, 低层 blocks 正好排在前面, 所以判断会更自然。

一旦找到了这个 k , 定位元素就非常容易。因为 level k 的 block 大小为 B^k , 所以该元素在 level k 中的相对偏移为

$$x - N_k.$$

于是它所在的 block 编号和 block 内偏移分别可以由除法和取模得到。由于论文中取 $B = 2^b$, 所以 B^k 也是二的幂, 除法和取模都可以用 shift 和 mask 在常数时间内完成。因此, 真正困难的地方只剩下一个: 如何在 $O(1)$ 时间内找到满足 $N_k \leq x < N_{k+1}$ 的层数 k 。

为此, 作者首先将 x 写成标准 base- B 表示, 并令 ℓ 为其最高非零位。由于 $B = 2^b$, 这个最高非零位可以通过机器字上的最高有效位操作得到:

$$\ell = \left\lfloor \frac{\text{msb}(x)}{b} \right\rfloor.$$

这里 $\text{msb}(x)$ 表示 x 的最高有效 bit 的位置。也就是说, ℓ 大致刻画了 x 的数量级。

接下来需要说明, 找到 ℓ 后, 真正的层数 k 不会离 ℓ 太远。论文利用每个 $n_j \leq 2B$ 这一性质, 得到

$$N_{\ell-1} = \sum_{j=0}^{\ell-2} n_j B^j \leq 2B \sum_{j=0}^{\ell-2} B^j \leq 3B^{\ell-1} \leq B^\ell \leq x.$$

因此有

$$x \geq N_{\ell-1},$$

从而可以推出

$$k \geq \ell - 1.$$

这一步非常关键。它说明我们不需要从所有层开始找，而是只需要从 $\ell - 1$ 附近开始判断。

此时可以分三种情况讨论。第一，如果

$$x < N_\ell,$$

那么结合 $N_{\ell-1} \leq x$ ，可知

$$N_{\ell-1} \leq x < N_\ell,$$

因此

$$k = \ell - 1.$$

第二，如果

$$N_\ell \leq x < N_{\ell+1},$$

那么直接得到

$$k = \ell.$$

第三，如果

$$N_{\ell+1} \leq x,$$

说明 x 已经越过了 level ℓ 对应的范围。此时需要继续向更高层寻找下一个非空层。也就是说， $k = \ell'$ ，其中 $\ell' > \ell$ ，并且 ℓ' 是满足 $n_{\ell'} > 0$ 的最小下标。

最后的问题就是：如何在 $O(1)$ 时间内找到这个下一个非空层。由于每个 n_j 可能达到 $2B$ ，论文将它拆成两部分：

$$n_j = n_j^0 + Bn_j^1,$$

其中

$$0 \leq n_j^0 < B, \quad 0 \leq n_j^1 \leq 2.$$

然后将所有低位部分打包成一个机器字：

$$N_0 = (n_{r-1}^0, \dots, n_1^0, n_0^0)_B,$$

再将所有高位部分打包成另一个机器字：

$$N_1 = (n_{r-1}^1, \dots, n_1^1, n_0^1)_B.$$

这样，某一层 j 非空当且仅当 $n_j^0 > 0$ 或 $n_j^1 > 0$ 。因此，只要观察 $N_0 \vee N_1$ 中对应 digit 是否非零，就可以判断某一层是否非空。

为了找到 ℓ 以上的第一个非空层，只需要用 mask 去掉低于等于 ℓ 的部分，然后找剩余部分中的最低有效位即可。由于最低有效位 lsb 也可以通过机器指令或由 msb 间接得到，这一步也可以在常数时间内完成。由此，作者成功在 $O(1)$ 时间内找到了目标元素所在的 level k 。

得到 k 后，再根据 N_k 计算层内偏移，并用 shift/mask 得到 block 编号和 block 内位置，就可以直接访问该元素。修改操作与访问操作完全相同，只是在定位到对应位置后将该位置的值替换掉。因此，access 和 modify 都可以支持 worst-case $O(1)$ 时间。

总的来说，这一部分的核心思想是：朴素扫描层级需要 $O(r)$ 时间，而论文通过把各层 block 数量编码成冗余 base- B 表示，再利用机器字上的 msb、mask、shift 等操作，将寻找元素所在层压缩为常数时间。换句话说，访问与修改之所以能做到 $O(1)$ ，关键不在于多层结构本身简单，而在于作者把层定位问题转化成了一个机器字上的位运算问题。

5.2 成员 B：下界分析

第 3 章已经说明，若稳定状态下的额外空间为 $s(N)$ ，扩缩容过程中允许的临时额外空间为 $t(N)$ ，则二者满足

$$s(N)t(N) = \Omega(N).$$

这一结论刻画的是空间之间的权衡。本节转而讨论更新时间：当稳定额外空间被压缩到 $O(rN^{1/r})$ 时，Grow 的均摊代价是否还可能低于 $O(r)$ 。论文第 8、9 节通过 growth game 证明，在一类标准实现中答案是否定的[1]。

5.2.1 标准实现及下界的适用范围

论文把块迁移遵循特定形式的结构称为 standard implementation。每当结构申请一个新块 B 时，它立即将若干旧块 $A_1, A_2, \dots, A_q$ 中的元素按顺序复制到 B ；一个旧块复制完毕后随即释放。此次合并结束后，结构重新回到只占用 $N + s(N)$ 空间的稳定状态。因而，临时空间只在“申请新块并搬移旧块”的过程中短暂存在。

这一限制使真实的块合并能够被组合游戏刻画。论文前面给出的分层结构以及经典分块结构都属于此类实现，但该定义并不直接覆盖把一次迁移长期分散到许多操作、使用更复杂编码或不遵循上述复制方式的算法。因此，后面的时间下界必须连同 standard implementation 这一前提一起理解。

5.2.2 (N, k, ℓ) -growth game

growth game 依次插入 N 个元素，并用 k 个初始为空的子数组 $A_1, A_2, \dots, A_k$ 保存它们。空子数组位于序列前部；第一个非空子数组至多保留 ℓ 个空位，其余非空子数

组均为满块。若 a_i 表示 A_i 中的元素数，则一次插入分为三种情况：

1. 第一个非空子数组仍有空位，直接写入新元素，代价为 1；
2. 所有非空子数组都已填满但仍有空子数组，申请一个大小为 $\ell + 1$ 的新块，写入一个元素并留下 ℓ 个空位，代价为 1；
3. 所有 k 个子数组都非空且已填满，选择某个 $i \in [k]$ ，将 $A_1, \dots, A_i$ 与新元素合并成新的 A_i ，并令前 $i - 1$ 个子数组重新为空。其代价为

$$1 + \sum_{j=1}^i a_j,$$

即一次新元素写入加上所有被搬移旧元素的赋值次数。

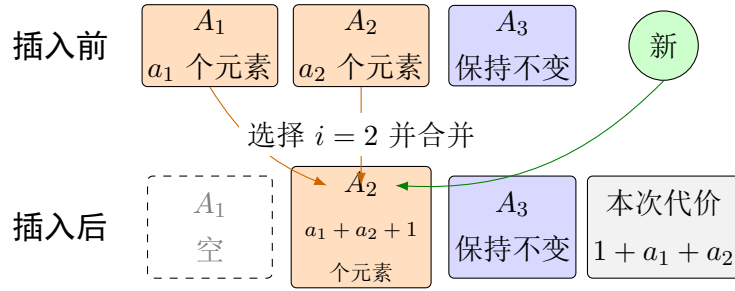


图 3: growth game 中选择 $i = 2$ 的一次合并示例（根据文献 [1] 整理）。

游戏忽略内存申请、释放和指针更新的成本，也不限制一次操作的临时空间，因此它给算法提供了比真实模型更宽松的条件。记完成游戏的最小总代价为 $C_{N,k,\ell}$ ，对应的最小均摊代价为

$$A_{N,k,\ell} = \frac{C_{N,k,\ell}}{N}.$$

5.2.3 二项式阈值与游戏下界

Lemma 8.2 首先消去空闲位置参数[1]。当 N 能被 $\ell + 1$ 整除时，连续的 $\ell + 1$ 次插入可视为一次成组插入，从而

$$C_{N,k,\ell} = (\ell + 1)C_{N/(\ell+1),k,0}, \quad A_{N,k,\ell} = A_{N/(\ell+1),k,0}.$$

因此，允许一个块保留少量空位只会缩短等价游戏的长度，并不会从根本上改变均摊搬移代价。以下简记 $C_{N,k} = C_{N,k,0}$ 和 $A_{N,k} = A_{N,k,0}$ 。

Theorem 8.6 精确求解了 $\ell = 0$ 的游戏[1]。若整数 $n \geq 0$ 满足

$$\binom{n+k-1}{k} - 1 \leq N \leq \binom{n+k}{k} - 1,$$

则

$$C_{N,k} = (N+1)n - \binom{n+k}{k+1}.$$

这里的 n 可以看作最优策略不可避免的合并深度：当插入数量跨过二项式阈值时，元素必须经历更多层合并。该定理的完整证明通过归纳刻画所有最优终态；对于后续下界，更关键的是 Corollary 8.13：若

$$\binom{n+k-1}{k} \leq N \leq \binom{n+k}{k} - 1,$$

则

$$A_{N,k} \geq \frac{k}{k+1}(n-1) \geq \frac{n-1}{2}.$$

换言之，只要游戏规模越过参数 n 对应的二项式阈值，就能得到 $\Omega(n)$ 的均摊代价。

5.2.4 从真实结构到 $\Omega(r)$ 下界

考虑稳定状态只使用

$$N + O(rN^{1/r})$$

空间的标准可变长数组，并令 $m = rN^{1/r}$ ；常数因子可通过微调参数吸收。稳定额外空间限制了结构能够同时维护的块数、块指针数和空闲位置，因此其操作可以映射到扩展的 (N, m, m) -growth game。扩展规则甚至允许合并任意一组子数组。Lemmas 8.17 和 8.18 说明这些额外自由不会降低最优总代价，所以该游戏覆盖了标准实现可能采用的块合并方式。

利用 Lemma 8.2，并忽略不影响渐近结果的整除和取整问题，可将游戏约化为

$$N' = \Theta\left(\frac{N}{m}\right) = \Theta\left(\frac{1}{r}N^{1-1/r}\right)$$

次成组插入、 $k = m = rN^{1/r}$ 个子数组且 $\ell = 0$ 的游戏。现在取 $n = \beta r$ ，其中 $\beta = 1/2$ 。由标准二项式估计

$$\binom{x}{y} \leq \left(\frac{ex}{y}\right)^y$$

可得

$$\binom{\beta r + rN^{1/r}}{\beta r} \leq \left(\frac{e(\beta + 1)}{\beta}\right)^{\beta r} N^{\beta}.$$

当 $r = O(\log N)$ 且 N 充分大时，取合适的常数即可使该上界小于 N' 。例如 $\beta = 1/2$ 时，证明中使用

$$r(3e)^{r/2}N^{1/2} \leq N^{3/4} \leq N^{1-1/r}.$$

因此，约化后的游戏已经越过 $n = \Theta(r)$ 对应的二项式阈值。由 Corollary 8.13，

$$A_{N',k} \geq \frac{n-1}{2} = \Omega(r).$$

这就得到 Theorem 9.1: 任何稳定存储空间为 $N + O(rN^{1/r})$ 、且 $r = O(\log N)$ 的标准可变长数组, 其 `Grow` 操作的均摊代价均为 $\Omega(r)[1]$ 。

第 6、7 节构造达到 $O(r)$ 均摊扩缩容时间, 因此该上界在标准实现范围内是渐近最优的。特别地, 第 7 节最终结构只使用更小的 $O(N^{1/r})$ 稳定额外空间, 自然也满足上述空间限制。需要强调的是, 这里证明的是标准实现的均摊下界, 而不是所有可能算法的无条件下界, 也不能直接解释为一般的最坏情况时间不可能性; 将结论推广到非标准算法仍是论文留下的问题。

6 技术直觉与方法评价

6.1 成员 A: 技术直觉与示例解释

6.1.1 技术直觉

事实上, 在这篇工作之前, 没有区分过具体使用什么空间, 最经典的 Brodnik 结论指出某些时刻会用到 $N + \Omega(\sqrt{N})$ 空间。

本文对这种空间进行细化, 没有否认这种结论, 而是指出了一个更一般性的规律, 这是作者的深刻洞见。这是从特殊到一般的归纳思维。

所以这引出来了**第一个技术直觉**, 就是旧下界中的 $\sqrt{N}$ 只是平时空间和临时空间相等的一个平衡点。问题的本质, 作者在这里指出来, 是一个乘积约束: $s(N)t(N) \geq N$ 。当 $s(N) = t(N) = \sqrt{N}$ 时, 正好回到 Brodnik 等人的结论; 而如果希望平时存储空间进一步下降, 比如 $s(N) = N^{1/r}$, 那么临时 `resize` 空间就必须上升到 $t(N) = N^{1-1/r}$ 。这说明所谓最优空间并不是绝对的, 而取决于我们如何定义和区分资源。

第二个技术直觉, 是作者发现 `resizable array` 中的元素并不是完全同质的。数组只在末尾 `grow` 和 `shrink`, 因此靠近末尾的元素是最新的、最容易被删除和改变的; 而靠前的元素更老, 也更稳定。本文的多层 `block` 构造正是利用了这种差异: 新加入的元素先放在小 `blocks` 中, 以便处理频繁的尾部变化; 当这些元素逐渐变得稳定之后, 再被合并进更大的 `blocks` 中, 从而减少 `block` 数量和指针开销。这和之前的证明一样, 就是中间证明的发现: 新加入的元素都在低层或者同层后面, 这正是作者发现了这一特性而设计出来的, 从这个角度观察, 之前的数据结构 (多层更新) 其实并不突兀, 只是利用了 `grow/shrink` 只发生在末尾这一操作特性。

可以具体解释一下这个直觉是怎么转化为数据结构的: 大小为 $B, B^2, \dots, B^{r-1}$ 的 `blocks` 对应不同的稳定层次。Level 1 的 `size-B blocks` 变化最频繁, 类似计数器中的低位; 更高层的 `blocks` 只有在低层积累到一定程度后才会改变, 类似计数器中的高位。因此, `base-B counter` 在这里解释了为什么这种分层结构是自然的: 低位频繁变化, 所以用小块; 高位很少变化, 所以用大块。作者实际上是用计数器的进位结构刻画了元素稳定性的变化过程。

第三个技术直觉，是本文对冗余的理解很深。但在本文中，冗余是防止 grow 和 shrink 在边界附近反复震荡的缓冲机制。对于 shrink 来说，如果每一层刚好满 B 个 blocks 就合并，刚好不够就拆分，那么 grow/shrink 交替出现时，就可能刚合并完又立刻拆开，刚拆开又马上合并，导致大量无意义的数据搬移。本文使用 redundant base- B counter，让每一层最多允许 $2B$ 个 blocks。

具体为什么选择 $2B$ ？我们这里可以给出一个回答：若一层达到 B 个 blocks 就立即合并，则合并后该层可能被清空，随后 shrink 就可能迫使刚生成的高层 block 被重新拆开。相比之下，本文 redundant base- B counter：当某层达到 $2B$ 个 blocks 时，只将其中 B 个合并到上一层，剩余 B 个 blocks 仍保留在当前层。因此，若一次合并生成了 size- B^k block，则低层仍至少保留 B^k 个元素；在下一次同级 split 发生前，必须至少经过 B^k 次 shrink。这一缓冲保证了分裂所需的复制成本和 credit 补充这两个需求可以由此前的 shrink 操作摊还支付。

6.1.2 示例解释

由于本部分插入图片和解释过多，不宜在 $latex$ 展示，所以请见附录，附录文档有详细的、直观的数据结构展示与思想解释。附录文档在 notes/A.notes/直观演示.md

6.2 成员 B：创新点、局限性与后续影响

前文已经分别介绍了论文的分层构造和下界证明。本节不再重复具体操作，而是从模型、理论完整性和实际可实现性三个角度评价其贡献与边界[1]。

6.2.1 核心创新与理论贡献

本文最重要的贡献首先是重新定义了“空间最优”的讨论方式。传统的 $N + \Omega(\sqrt{N})$ 下界把稳定状态下的存储空间和单次扩缩容过程中短暂出现的临时空间合并计算，容易使人误以为两者都不能低于 $\sqrt{N}$ 。Tarjan 和 Zwick 将它们分别记为 $s(N)$ 和 $t(N)$ ，并证明了如下乘积下界[1]：

$$s(N)t(N) = \Omega(N).$$

这一模型揭示的不是单一最优点，而是一条空间 trade-off 曲线：稳定空间可以进一步下降，只要允许 resize 时短暂申请更大的块。

其次，论文把这一模型转化为可达到的参数化结果。对整数参数 r ，分层块结构与 data structuring transformation 共同实现

$$s(N) = O(N^{1/r}), \quad t(N) = O(N^{1-1/r}),$$

同时保持 $O(1)$ 最坏情况访问和 $O(r)$ 均摊扩缩容时间。 r 因而不只是证明中的层数，也直接表达设计者在稳定空间、临时峰值和更新代价之间的选择。与只给出某个固定结构相比，这一族构造更完整地描述了问题的可行区域。

最后，论文没有停留在上界构造，而是引入 growth game 对块合并过程进行抽象，并精确求解游戏的最优总代价。由此得到的 $\Omega(r)$ 均摊下界与构造的 $O(r)$ 上界匹配。空间乘积下界和 growth game 时间下界分别封闭了 trade-off 的两个方向，使“最优”不仅是对某个算法分析得到的结论，而是在明确模型中的渐近最优性结论。

6.2.2 模型假设与定理边界

上述最优性依赖若干需要明确保留的前提。首先， $O(1)$ 访问建立在 word-RAM 模型上。第 6.3 节把各层块数压入少数机器字，并借助 `msb`、`lsb`、位掩码和移位在常数时间定位目标层；常数时间计算 `msb` 的一种方案还依赖乘法。作者也指出，对实际中预期很小的 r ，直接扫描 $O(r)$ 层或进行 $O(\log r)$ 二分可能比位级实现更合适[1]。因此，这里的常数访问时间主要完善了理论界，并不自动意味着对应实现具有更低的工程开销。

其次，最终的 Theorem 7.3 并非对任意增长速度的 $r(N)$ 都直接成立，其参数范围为

$$r \leq \frac{1}{2} \frac{\log N}{\log \log N}.$$

该定理还需要把第 6 节参数为 $2r$ 的标准结构作为转换输入[1]。若只陈述最终复杂度而省略范围和参数替换，会把论文实际证明的结果扩大。

更关键的限制来自时间下界。Theorem 9.1 只覆盖 standard implementations，即新块申请后立即复制旧块并在操作结束时恢复稳定空间界的实现。作者相信，在对元素和指针加入适当的不可压缩性假设后，下界可以推广到所有算法，但论文没有给出这一推广的形式化证明[1]。因此，“非标准算法不会更好”属于作者的判断，而不是本文已经证明的定理。

6.2.3 均摊更新与工程可行性

论文的 Grow 和 Shrink 时间是 $O(r)$ 均摊界，而不是最坏情况界。对此也应区分已证明的下界和结构本身的空间障碍：Theorem 9.1 的 $\Omega(r)$ 均摊下界并不排除一个假想的 $O(r)$ 最坏情况算法；现有构造难以直接去摊还化，是因为常规 background rebuilding 会改变空间的归类。

在最高层合并中，新块规模可达

$$\Theta(N^{1-1/r}).$$

若复制在一次操作内完成，该块只计入临时空间；若把复制分散到多个操作，新旧表示会跨操作同时保留，未完成的新块便必须计入稳定空间。当 $r > 2$ 时，

$$N^{1-1/r} > N^{1/r},$$

所以常规后台重建需要持久保存的块可能超过 $O(N^{1/r})$ 的稳定额外空间预算。这说明标准 background rebuilding 不能直接用于第 6、7 节结构，但不能据此断言所有非标准去摊还化方案都不存在。 $r = 2$ 时两种规模同为 $\Theta(\sqrt{N})$ ，也与 HAT 和 Brodnik 等经典结构能够获得最坏情况更新界的事实一致[7, 8]。

从实现角度看，多层块目录、rebuild 阈值、冗余 base- B 计数器和位操作都会带来额外常数与维护复杂度。论文的优势主要体现在稳定额外空间极为受限的理论场景；在一般程序中，连续内存的缓存局部性、分配器成本和实现复杂度可能使简单的几何扩容或较浅的分块结构更有吸引力。这不削弱渐近结果，但说明理论最优与工程选择采用的是不同评价标准。

6.2.4 开放问题与后续意义

论文结尾明确留下两个理论问题[1]。第一，将 $\Omega(r)$ 均摊下界从 standard implementations 推广到所有可能的可变长数组算法，并为元素与指针的不可压缩性建立合适模型。第二，growth game 的精确解虽然完整，但证明较长；是否存在不求精确值、只用更短的归纳或势能论证就能推出所需渐近下界，仍值得研究。

从评述角度，还可以进一步考察固定小参数下的实际表现，例如比较 $r = 2$ 、 $r = 3$ 分层结构与 HAT、几何扩容数组在缓存行为、分配次数和常数开销上的差异。无论这些结构是否直接进入工程实现，本文都改变了该问题的理论视角：它把“最省空间的动态数组”从寻找单个结构，转化为刻画稳定空间、临时空间和更新时间三者的完整可达边界；growth game 也提供了一种分析受限块合并策略的独立组合工具。

7 总结

7.1 成员 B：总结

Tarjan 和 Zwick 的工作重新审视了可变长数组中“空间最优”的含义。传统分析通常只记录扩缩容期间出现过的最大空间占用，因此得到 $N + \Theta(\sqrt{N})$ 的经典界限；本文则把稳定状态下用于存储和访问的额外空间 $s(N)$ ，与一次 Grow 或 Shrink 中短暂使用的临时空间 $t(N)$ 分开计算。这个模型更准确地反映了同时维护许多数组、但只有少数数组正在扩缩容的场景，也把问题从寻找单一最优结构转化为刻画稳定空间、临时空间和更新时间之间的完整折中关系[1]。

在上界方面，论文以整数参数 r 控制分层深度，通过大小为 $B, B^2, \dots, B^{r-1}$ 的数据块、冗余 base- B 计数器和后续的 data structuring transformation，得到稳定存储空间 $N + O(N^{1/r})$ 、扩缩容峰值空间 $N + O(N^{1-1/r})$ 的可变长数组。该结构同时支持 $O(1)$ 最坏情况时间的 **Get** 和 **Set**，以及 $O(r)$ 均摊时间的 **Grow** 和 **Shrink**。 $r = 2$ 对应经典的平方根级额外空间；增大 r 则能够进一步压缩长期保留的空间，但需要更深的分层和更高的均摊更新代价。

论文还从空间和时间两个方向说明这些结果的最优性。对一般实现，块数量与大块分配之间的矛盾推出 $s(N)t(N) = \Omega(N)$ ，因此上述稳定空间和临时空间的指数折中已经达到渐近最优。对 standard implementations，作者进一步把真实的块合并映射为 growth game，并利用二项式阈值证明：当稳定空间限制为 $N + O(rN^{1/r})$ 且参数满足相应条件时，**Grow** 的均摊代价至少为 $\Omega(r)$ 。这一结论与构造的 $O(r)$ 上界匹配，但其适用范围不能直接扩展到所有可能的非标准算法。

总体而言，本文最重要的贡献不仅是给出一个更省稳定空间的数组实现，还在于建立了一套较完整的时间-空间边界：乘积下界刻画空间折中，分层构造给出可达上界，growth game 则解释标准块合并策略为何必须付出相应的更新时间。与此同时，word-RAM、常数时间位操作、均摊更新以及 standard implementation 等假设也限制了结论的直接工程意义。如何把时间下界推广到一般算法、简化 growth game 的下界证明，并评估固定小参数结构的实际表现，仍是值得继续研究的问题。

参考文献

- [1] Robert E. Tarjan and Uri Zwick. Optimal resizable arrays, 2023.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [3] Phil Bagwell. Fast functional lists. In *Implementation of Functional Languages*, volume 2670 of *Lecture Notes in Computer Science*, pages 34–50. Springer, 2003.
- [4] Michael T. Goodrich and John G. Kloss II. Tiered vectors: Efficient dynamic arrays for rank-based sequences. In *Algorithms and Data Structures*, volume 1663 of *Lecture Notes in Computer Science*, pages 205–216. Springer, 1999.
- [5] Philip Bille, Anders Roy Christiansen, Mikko Berggren Ettienne, and Inge Li Gørtz. Fast dynamic arrays. In *25th Annual European Symposium on Algorithms (ESA 2017)*, volume 87 of *Leibniz International Proceedings in Informatics*, pages 16:1–16:13. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2017.

- [6] Stelios Joannou and Rajeev Raman. An empirical evaluation of extendible arrays. In *Experimental Algorithms*, volume 6630 of *Lecture Notes in Computer Science*, pages 447–458. Springer, 2011.
- [7] Edward Sitarski. Algorithm alley: Hats: Hashed array trees. *Dr. Dobb's Journal of Software Tools*, 21(9):107, 1996.
- [8] Andrej Brodnik, Svante Carlsson, Erik D. Demaine, J. Ian Munro, and Robert Sedgewick. Resizable arrays in optimal time and space. In *Proceedings of the 6th International Workshop on Algorithms and Data Structures (WADS)*, pages 37–48, 1999.